

Data Science project in Python

Description:

1. Introduction

The **Expense Tracker** is a Python-based command-line application designed to help users record, view, categorize, analyze, and delete their daily expenses. The project stores expense information in a **JSON file**, allowing the data to remain available even after the program is closed.

2. Objective

The main objectives of this project are:

- To record daily expenses.
- To organize expenses according to categories such as Food, Travel, and Books.
- To view all recorded expenses.
- To calculate the total expense for each category.
- To calculate the overall expenditure.
- To delete unwanted expense records.
- To store expense data permanently using a JSON file.

3. Technologies Used

- **Programming Language:** Python
- **Data Storage:** JSON
- **Interface:** Command Line Interface (CLI)
- **Python Modules:** `json`, `os`, and `datetime`

4. Working of the Project

The application displays a menu with five options:

1. **Add Expense**
2. **View All Expenses**
3. **View Totals by Category**
4. **Delete an Expense**
5. **Exit**

The program continuously displays this menu and performs the operation selected by the user.

5. Adding an Expense

When the user selects **Add Expense**, the program accepts:

- Amount
- Category
- Optional note

It automatically records the current date and stores the information as an expense entry. The entry is then added to the expense list and saved to the JSON file.

6. Viewing Expenses

The **View All Expenses** option displays the recorded expenses in a table-like format containing the **date, category, amount, and note**. If there are no expenses, the program displays an appropriate message.

7. Expense Analysis

The project can calculate expense totals based on categories. It adds the amounts belonging to each category and displays the category-wise totals. It also calculates the **grand total** of all recorded expenses.

8. Deleting an Expense

The user can delete an expense by entering its row number. The selected record is removed from the expense list, and the updated data is saved back to the JSON file.

9. JSON Data Storage

The project uses `expenses.json` to store expense records. When the program starts, it loads existing data from this file. Whenever an expense is added or deleted, the updated list is saved back to the file. This provides **persistent data storage** between program executions.

CODE:

```
"""
```

```
EXPENSE TRACKER (Python)
```

```
-----
```

```
A command-line expense tracker that lets you add, view, and analyze  
expenses by category, with data persisted to a JSON file.
```

```
Run:  python expense_tracker.py
```

```
"""
```

```
import json
```

```
import os
```

```

from datetime import date

DATA_FILE = "expenses.json"

def load_expenses():
    if not os.path.exists(DATA_FILE):
        return []
    with open(DATA_FILE, "r") as f:
        try:
            return json.load(f)
        except json.JSONDecodeError:
            return []

def save_expenses(expenses):
    with open(DATA_FILE, "w") as f:
        json.dump(expenses, f, indent=2)

def add_expense(expenses):
    try:
        amount = float(input("Amount: "))
    except ValueError:
        print("Invalid amount. Please enter a number.")
        return

    category = input("Category (e.g. Food, Travel, Books): ")
    category = category.strip().title()
    note = input("Note (optional): ").strip()

    entry = {
        "date": str(date.today()),
        "amount": amount,
        "category": category,
        "note": note,
    }
    expenses.append(entry)
    save_expenses(expenses)
    print(f"Added: {amount} under '{category}'.")

```

```

def view_expenses(expenses):
    if not expenses:
        print("No expenses recorded yet.")
        return

    print(f"\n{'Date':<12}{'Category':<15}{'Amount':<10}Note")
    print("-" * 55)
    for e in expenses:

print(f"{e['date']:<12}{e['category']:<15}{e['amount']:<10}{e['note']}")

def view_by_category(expenses):
    if not expenses:
        print("No expenses recorded yet.")
        return

    totals = {}
    for e in expenses:
        totals[e["category"]] = totals.get(e["category"], 0) + e["amount"]

    print("\n--- Totals by Category ---")
    for category, total in sorted(totals.items(), key=lambda x: -x[1]):
        print(f"{category:<15} {total:.2f}")

    grand_total = sum(totals.values())
    print("-" * 25)
    print(f{'TOTAL':<15} {grand_total:.2f}")

def delete_expense(expenses):
    view_expenses(expenses)
    if not expenses:
        return
    try:
        index = int(input("\nEnter the row number to delete (1-based): "))
- 1
        if 0 <= index < len(expenses):

```

```

        removed = expenses.pop(index)
        save_expenses(expenses)
        print(f"Deleted: {removed}")
    else:
        print("Invalid index.")
except ValueError:
    print("Please enter a valid number.")

def main():
    expenses = load_expenses()

    menu = """
===== EXPENSE TRACKER =====
1. Add Expense
2. View All Expenses
3. View Totals by Category
4. Delete an Expense
5. Exit
"""
    while True:
        print(menu)
        choice = input("Enter choice: ").strip()

        if choice == "1":
            add_expense(expenses)
        elif choice == "2":
            view_expenses(expenses)
        elif choice == "3":
            view_by_category(expenses)
        elif choice == "4":
            delete_expense(expenses)
        elif choice == "5":
            print("Goodbye! Your data is saved in expenses.json.")
            break
        else:
            print("Invalid choice, try again.")

if __name__ == "__main__":

```

```
main()
```

OUTPUT:

```
===== EXPENSE TRACKER =====
... 1. Add Expense
     2. View All Expenses
     3. View Totals by Category
     4. Delete an Expense
     5. Exit

Enter choice: 1
Amount: 25000
Category (e.g. Food, Travel, Books): food
Note (optional): pizza
Added: 25000.0 under 'Food'.

===== EXPENSE TRACKER =====
1. Add Expense
2. View All Expenses
3. View Totals by Category
4. Delete an Expense
5. Exit

Enter choice: 2

Date          Category      Amount      Note
-----
2026-08-08    Food          25000.0     pizza

===== EXPENSE TRACKER =====
1. Add Expense
2. View All Expenses
3. View Totals by Category
4. Delete an Expense
5. Exit

Enter choice: 4

Date          Category      Amount      Note
-----
2026-08-08    Food          25000.0     pizza

Enter the row number to delete (1-based): 1
Deleted: {'date': '2026-08-08', 'amount': 25000.0, 'category': 'Food', 'note': 'pizza'}

===== EXPENSE TRACKER =====
1. Add Expense
2. View All Expenses
3. View Totals by Category
4. Delete an Expense
5. Exit

Enter choice: 
```

CONCLUSION:

The **Expense Tracker** demonstrates the practical use of Python programming concepts such as **functions, lists, dictionaries, loops, conditional statements, exception handling, file handling, JSON data storage, and date handling**. It provides a simple way to manage and analyze personal expenses through a command-line interface.